

테이블 머신러닝 정리

LG Aimers 온라인 교육과 해커톤을 거치며 정리한 개념 · 구현 · 시너지 · 실측 기록

기초 개념부터 실전 판정 규율까지 · 기법 52종 · 2026

이 책을 읽는 법

각 기법은 **일정한 형식**으로 서술한다. 급하면 **직관**과 **실측**만 읽어도 된다.

절	내용
직관	왜 이게 작동하는가. 수식 없이
형식	수식·알고리즘
구현	바로 쓸 수 있는 코드
장점 / 단점	무엇을 얻고 무엇을 감수하는가
언제	이 기법을 꺼낼 조건
 시너지	같이 쓰면 서로 를 강화하는 기법
 안티시너지	같이 쓰면 서로 를 무력화하거나 위험한 조합
실측	이 대회에서 나온 실제 숫자

채택 실전 점수를 냈다 **기각** 시도했고 실패 **보류** 여건이 안 됐다

구성: **0부**는 LG Aimers 온라인 교육(Tabular ML · 지도학습)에서 배운 **기초 개념**을 정리한 것이고, **1~11부**는 해커톤에서 **직접 측정하며 배운 것**이다. 기초를 아는 독자는 0부를 건너뛰어도 된다.

가장 중요한 한 줄: 이 대회에서 점수를 준 것은 **구조적 변경**(새 피쳐 블록 · 독립 파이프라인 결합 · 엔티티 록업)뿐이고, **미세조정은 전부 ±1~13점** 안에 있었다.

목차

0부 · 기초 개념

- 0.1 지도학습이란
- 0.2 결정 트리
- 0.3 과적합과 편향-분산
- 0.4 앙상블이란
- 0.5 배깅과 랜덤 포레스트
- 0.6 부스팅과 GBDT
- 0.7 세 라이브러리 비교
- 0.8 하이퍼파라미터

1부 · 토대

- 1.1 지표의 대수적 구조
- 1.2 노이즈 바닥
- 1.3 자유도라는 통화

2부 · 모델링

- 2.1 GBDT
- 2.2 신경망
- 2.3 시드 배경
- 2.4 목적함수 선택
- 2.5 지식 증류

3부 · 피쳐

- 3.1 as-of 분해
- 3.2 시즌 상대화
- 3.3 도메인 물리
- 3.4 잠재변수
- 3.5 숨은 라벨 디코딩
- 3.6 상태 복원

4부 · 엔티티 록업

- 4.1 타겟 인코딩과 EB

4.2 이중 중심화 잔차 록업

4.3 대비 타겟

4.4 축 스윙

4.5 실패한 변형들

5부 · 캘리브레이션

5.1 아핀 보정

5.2 로짓 셀 오프셋

5.3 LB 역산

5.4 분산 수축 분해

5.5 고자유도 사후처리의 함정

6부 · 앙상블

6.1 2차형식

6.2 최적 가중치

6.3 ρ

6.4 게이팅

6.5 스택킹

6.6 라이브러리 천장

7부 · 검증

7.1 폴드와 예측 시계

7.2 레짐

7.3 중첩

7.4 사전 등록

7.5 무작위 대조군

7.6 시드

7.7 전이비

7.8 상한 스크린

8부 · 규정·누출

8.1 행 독립성

8.2 누출 유형

8.3 특권 피처

9부 · 시너지 지도

9.1 시너지 매트릭스

9.2 핵심 결합 5종

9.3 금지 조합 7종

10부 · 운영

10.1 제출물

10.2 GPU·클라우드 연산

10.3 로컬 실행 인프라

10.4 다중 에이전트

11부 · 실행 순서

0부 · 기초 개념 — 무엇이 무엇인가

LG Aimers 온라인 교육(Tabular ML · 지도학습)에서 배운 내용을 정리했다. 1부부터는 이 기초 위에서 **해커톤에서 직접 측정한 것**을 다룬다. 이미 아는 독자는 **0.5 배깅의 상관 효과**와 **0.6 GBDT**만 보고 넘어가도 된다.

0.1 지도학습이란

직관 입력 x 와 정답 y 의 쌍을 잔뜩 보여주고, **처음 보는 x 에 대해 y 를 맞히는 함수**를 찾는 것이다. "이 투구는 제구에 성공할까?"처럼 답이 정해져 있는 문제가 여기 속한다.

구분	출력	예	대표 지표
분류	범주 (또는 각 범주의 확률)	성공/실패, 개/고양이	정확도, AUC, LogLoss
회귀	연속값	가격, 수요량	MSE, MAE

이 책이 다루는 문제는 **확률을 출력하는 이진 분류**다. "성공/실패"를 맞히는 게 아니라 **성공 확률을 정확히 대는 것**이 목표라 평가도 정확도가 아니라 **Brier**(확률의 제곱오차)를 쓴다.

왜 이 구분이 중요한가 지표가 확률 기반(Brier/MSE)이면 **1.1의 2차형식 성질**이 성립해 앙상블 계산이 대수 문제로 바뀐다. 정확도·AUC였다면 이 책의 절반은 못 썼다.

0.2 결정 트리 — 모든 것의 출발점

직관 스무고개다. "너비 > 6.5cm?" 같은 질문으로 데이터를 계속 둘로 나누고, 더 이상 나누지 않는 마지막 칸(**잎 노드**)에 도착하면 **그 칸에 속한 학습 데이터의 평균**을 답으로 낸다.

형식

$f(x) = \sum_j \hat{y}_j \cdot 1(x \in R_j)$ # R_j = j 번째 잎이 담당하는 영역
 $\hat{y}_j = (R_j \text{ 안 } y \text{들의 평균})$ # 분류면 다수결 / 클래스 비율

어떻게 나눌 곳을 정하나

최적 분할을 찾는 것은 **NP-complete**이고 목적함수가 미분 불가능하다. 그래서 **탐욕적(greedy)**으로 한 번에 한 노드씩 키운다 — 가능한 (노드 k , 피쳐 d , 임계값 t) 조합 중 **손실이 가장 많이 줄어드는 것**을 고른다.

손실 $L(\theta) = \sum_j p(j) \cdot \ell(j)$ $p(j)$ =그 잎에 도달할 확률, $\ell(j)$ =그 잎의 기대손실

분류 손실: Gini $\ell(k) = \sum_c \hat{y}_{kc}(1 - \hat{y}_{kc})$
엔트로피 $\ell(k) = -\sum_c \hat{y}_{kc} \log \hat{y}_{kc}$

회귀 손실: $\ell(k) = \text{평균제곱오차}$

장점 규칙 기반이라 해석이 쉽다. 비선형과 피쳐 상호작용을 자동으로 잡는다. **스케일링·정규화가 필요 없다**.

단점 깊게 키우면 **과적합**. 불안정하다 — 데이터가 조금만 바뀌어도 전혀 다른 트리가 나온다.

이 불안정성이 오히려 **앙상블의 재료**가 된다(0.5). 트리 하나는 약하지만, 서로 다른 트리를 많이 모으면 강해진다.

0.3 과적합·과소적합과 편향-분산

직관 너무 얕으면 중요한 패턴을 놓치고(과소적합), 너무 깊으면 학습 데이터의 잡음까지 외운다(과적합). 완전히 키운 트리는 학습 정확도가 거의 100%인데 새 데이터에서는 형편없다.

제어 방법	내용	대표 파라미터
사전 가지치기	너무 복잡해지기 전에 멈춘다	<code>max_depth</code> <code>min_samples_leaf</code> <code>min_impurity_decrease</code>
사후 가지치기	크게 키운 뒤 불필요한 가지를 제거	비용-복잡도 가지치기

편향(bias)은 모델이 구조적으로 못 맞히는 부분, 분산(variance)은 학습 데이터가 바뀔 때 예측이 흔들리는 정도다. 얇은 트리는 편향이 크고, 깊은 트리는 분산이 크다. 양상블의 두 갈래가 각각 이 둘을 공략한다:

배깅 → 분산을 줄인다 (편향은 그대로)
부스팅 → 편향을 줄인다 (분산은 관리해야 함)

0.4 양상블이란

직관 여러 모델의 예측을 합쳐 하나보다 잘 일반화하게 만든다. 사람 여럿에게 물어보는 것과 같은데, 다들 똑같이 생각하면 물어볼 이유가 없다.

따라서 모델들은 어떻게든 달라야 한다:

- 다른 데이터로 학습 (→ 배깅)
- 다른 알고리즘 (→ GBDT + 신경망)
- 다른 하이퍼파라미터 · 다른 시드

이 책 전체의 주제로 이어진다 "얼마나 달라야 하는가"를 정확히 재는 것이 6.3의 ρ 다. 그리고 다른 것만으로는 부족하고 방향이 맞아야 한다는 것이 해커톤에서 배운 가장 반직관적인 사실이었다.

종류	학습 방식	노리는 것	예
배깅	병렬 — 서로 독립적으로	분산 감소	Random Forest
부스팅	순차 — 앞의 실수를 보고	편향 감소	AdaBoost, GBDT
스태킹	예측을 입력으로 하는 메타 모델	비선형 결합	—

0.5 배깅과 랜덤 포레스트 ★ 여기서 ρ 가 처음 나온다

배깅 (Bootstrap Aggregating)

1. 원본 데이터 n 개에서 복원추출로 n 개를 뽑는다 → 이걸 m 번 반복
2. 각 데이터셋으로 모델을 독립적으로 학습
3. 예측을 평균 (분류면 다수결)

복원추출이라 각 데이터셋은 원본의 약 63%만 고유 샘플로 담는다 ($1 - 1/e \approx 0.632$). 남은 37%는 공짜 검증셋으로 쓸 수 있다 (OOB).

왜 좋아지나 — 그리고 왜 기대만큼은 아닌가

모델들이 완전히 독립이라면 분산이 $1/m$ 로 줄고 편향은 그대로다:

```
E[ŷ] = E[ŷi] # 편향 불변
Var[ŷ] = (1/m) · Var[ŷi] # 분산 1/m
```

그런데 실제로는 독립이 아니다. 같은 원본에서 뽑았으니 서로 닮는다. 예측들의 상관을 ρ 라 하면:

$$\text{Var}[\hat{y}] = (1/m)(1-\rho)\sigma^2 + \rho\sigma^2$$

두 번째 항은 m 을 아무리 키워도 사라지지 않는다. 모델을 100개 만들든 1000개 만들든, ρ 가 바닥을 만든다.

→ 그래서 배깅의 핵심은 개수가 아니라 상관을 낮추는 것이다.

랜덤 포레스트 — 상관을 한 번 더 낮추기

배깅에 두 번째 무작위성을 추가한다: 각 노드에서 분할을 고를 때 전체 피처가 아니라 무작위 부분집합만 본다. 그러면 강한 피처가 모든 트리를 지배하는 것을 막아 트리들이 더 달라진다.

이 개념이 실전에서 어떻게 이어졌나 위의 $\rho\sigma^2$ 항이 이 책 6.3의 출발점이다. 해커톤에서 나는 이 원리가 모델 간 블렌드에도 그대로 적용되고, 게다가 "상관을 낮추는 것" 자체가 목표가 되면 안 된다는 것까지 실측으로 확인했다 (노이즈를 주입해도 상관은 낮아지지만 점수는 떨어진다).

0.6 부스팅과 GBDT ★ 이 대회 주력

직관 배깅이 "여러 명에게 동시에 물어보기"라면, 부스팅은 "앞사람이 틀린 것만 골라 다음 사람에게 다시 물어보기"다. 약한 모델을 순차적으로 쌓아 남은 오차를 계속 줄여 나간다.

GBDT — 잔차를 학습한다

1. 초기 예측: $F_0(x) = \bar{y}$ (그냥 전체 평균)
2. 남은 오차: $r^{(m)}_i = y_i - F_{\{m-1\}}(x_i)$ ← 잔차
3. 트리 학습: $h_m(x)$ 를 잔차에 맞춘다
4. 앙상블 갱신: $F_m(x) = F_{\{m-1\}}(x) + \eta \cdot h_m(x)$

왜 "그라디언트" 부스팅인가

제곱오차 손실 $\frac{1}{2}(y - F)^2$ 를 F 로 미분하면 $-(y - F)$, 즉 잔차가 곧 손실의 그라디언트다. 그래서 "잔차를 맞추는 트리를 더한다"는 것은 함수 공간에서 경사하강을 한 걸음 내딛는 것과 같다. η (학습률)가 그 보폭이다.

왜 얇은 트리를 쓰나

- 비선형·상호작용을 별도 처리 없이 잡는다
- 잔차(의사 잔차)를 효율적으로 학습한다
- 앙상블에 구간별 상수 보정을 자연스럽게 더한다

단점 순차적이라 병렬화가 어렵고, 계속 오차를 쫓기 때문에 과적합에 민감하다. 튜닝이 배깅보다 까다롭다.

0.7 세 라이브러리 — 무엇이 다른가

셋 다 GBDT 원리는 동일하고, 다른 실용적 문제를 푼다.

라이브러리	핵심 초점	구체적으로 무엇이 다른가
XGBoost 2016	견고하고 확장 가능한 부스팅	정규화 항을 손실에 명시적으로 넣어 과적합을 억제. 결측 처리와 희소 데이터에 강하다. 가장 오래돼 레퍼런스가 많다
LightGBM 2017	빠르고 메모리 효율적인 학습	leaf-wise 성장 — 트리를 층별로 고르게 키우지 않고 손실이 가장 많이 줄어드는 잎부터 키운다. 같은 잎 개수로 더 깊이 파고들어 빠르고 정확하지만 과적합 위험이 크다(→ <code>num_leaves</code> 통제 필수)
CatBoost 2018	범주형 피처의 효과적 처리	순서형 부스팅(ordered boosting) — 타겟 인코딩 시 자기 행의 라벨을 쓰지 않도록 데이터에 가상의 순서를 부여해 누출을 막는다. 범주형이 많을 때 강하다

level-wise (XGBoost 기본)



leaf-wise (LightGBM)



실전에서는 "어느 게 제일 좋은가"는 데이터마다 다르다. 이 대회에서는 셋을 다 쓰고 블렌드했다(0.4의 "다른 알고리즘" 전략). 그리고 "권장 설정"이 항상 맞지는 않았다 — CatBoost의 범주형 자동 처리(`cat_features`)를 켜더니 오히려 -15.49 였다. 재보고 결정하라는 것이 이 책 전체의 태도다.

0.8 하이퍼파라미터 — 무엇을 왜 만지나

파라미터	올리면	주의
트리 개수 <code>n_estimators</code>	표현력 증가	과적합 · 학습시간 증가
학습률 <code>η</code>	빠르게 수렴	보수적이지 않아 과적합. 트리 개수와 반비례로 맞춘다
깊이 / 잎 개수	더 복잡한 상호작용	과적합. LightGBM은 <code>num_leaves</code> 가 핵심
서브샘플링	견고성 증가	너무 낮으면 학습 부족
정규화 <code>reg_lambda</code>	과적합 억제	너무 크면 과소적합
최소 잎 샘플수	과적합 억제	희소 구간을 못 잡음

⚠ 여기서부터 1부의 내용이 필요해진다. 하이퍼파라미터를 바꾸면 점수가 움직인다. 그런데 그 움직임이 개선인지 우연인지를 구별하려면 노이즈 바닥(1.2)을 먼저 알아야 한다. 이 대회에서 노이즈 바닥은 ± 12 점이었다고, 그보다 작은 "개선" 기록 다수가 사후에 무효화됐다.

0부 요약 — 1부로 넘어가기 전에

개념	한 줄
결정 트리	공간을 재귀적으로 쪼개고 각 칸의 평균을 답으로 낸다. 해석은 쉽고 불안정하다
과적합	학습 데이터의 잡음까지 외우는 것. 깊이·정규화로 통제한다
앙상블	여러 모델을 합친다. 서로 달라야 의미가 있다
배깅	독립 학습 후 평균 → 분산 감소. 단 $\rho\sigma^2$ 항이 바닥을 만든다
부스팅	잔차를 순차적으로 학습 → 편향 감소. GBDT가 대표
GBDT	$F_m = F_{m-1} + \eta \cdot h_m$, 잔차 = 손실의 그래디언트
세 라이브러리	원리는 같고 초점이 다르다 — 견고함 / 속도 / 범주형

1부부터는 여기서 배운 것을 실제 대회에서 재본 기록이다. 가장 중요한 연결은 0.5의 $\rho\sigma^2$ 항이다 — 그것이 6.3에서 "새 모델을 추가할 가치가 있는가"를 판정하는 유일한 수치가 된다.

1부 · 토대

이 세 개념은 나머지 전부의 전제다. 여기를 건너뛰면 뒤의 판정을 신뢰할 수 없다.

1.1 지표의 대수적 구조 채택

직관 대회 지표가 예측값에 대해 2차식이면, 여러 모델을 섞는 문제는 **포물선의 꼭짓점 찾기**가 된다. 포물선은 세 점만 알면 완전히 결정되므로, **모델을 실제로 섞어보지 않고 최적 조합을 계산**할 수 있다. 이 성질을 첫날 확인하면 이후 앙상블 실험의 대부분이 산수로 바뀐다.

형식

```
Brier skill = C · (1 - Brier/V), V = r(1-r), r = 라벨 평균
blend p =  $\sum w_i p_i$ ,  $\sum w = 1$ 
MSE(p) =  $\sum_{i,j} w_i w_j \cdot E[(p_i - y)(p_j - y)]$  ← w에 대해 정확히 2차

Gram 행렬:  $u(w) = w^T M w$ 
Mii = si
Mij = (si + sj)/2 + (C/2V) · dij2 dij = ||pi - pj||RMS
```

즉 n개 모델의 모든 조합 점수가 스킬 n개 + 쌍거리 n(n-1)/2개로 완전히 결정된다. 3-arm 이면 6개 수치다.

장점 실험을 계산으로 대체. 새 모델의 기여도를 **학습 전에** 추정 가능(6.3과 결합).

단점 AUC·LogLoss·F1에는 성립하지 않는다. LogLoss는 로짓 공간 근사만 가능.

🔗 시너지 0.5 배경의 상관 효과(같은 원리의 교과서판) · 6.3 p(이 식에서 직접 유도) · 6.6 라이브러리 천장(같은 행렬 재사용) · 7.8 상한 스크린(필요 ρ를 이 식으로 역산)

실측 실제 모델 5종의 관측 기여도를 **최대오차 0.02**로 재현. 이후 블렌드 실험을 전부 종이 계산으로 대체했다.

1.2 노이즈 바닥 채택

직관 같은 방법을 시도만 바꿔 두 번 돌리면 점수가 얼마나 흔들리는가. 이 폭을 모르면 "개선했다"와 "운이 좋았다"를 구별할 수 없다. 이 값은 이후 모든 판정의 단위가 된다.

구현

```
scores = [train_and_score(seed=s) for s in [0,1,2,3,4]] # 최소 5
floor = 2 * np.std(scores, ddof=1) # 이 값 미만의 차이는 개선이 아니다
```

⚠ 이 값은 **시드 노이즈**다. 결정적 사후처리(같은 입력이면 같은 출력)에는 시드 노이즈가 0이므로 그대로 갖다 쓰면 안 된다. 이 대회에서 실제로 오용한 적이 있다.

실측 두 제출물의 md5 전수 비교로 **신경망 서브모델만 다르고 나머지가 바이트 동일한 쌍**을 찾아 LB 차를 켜다 → ±12점. 이 값 미만의 "개선" 기록 다수가 사후에 무효화됐다.

1.3 자유도라는 통화 — 이 책의 숨은 주제

직관 어떤 기법이 **살아남는가 죽는가**는 대개 **자유도**가 결정한다. 파라미터가 적을수록 폴드를 넘어 전이되고, 많을수록 그 폴드의 특수사정을 외운다. 같은 검증을 통과해도 자유도가 다르면 운명이 다르다.

기법	자유도	정직 분할	실전
아핀 보정	2	통과	✅ +12.9
엔티티 잔차 룩업 (EB+게이트)	실효 낮음	통과	✅ +23.3
잔차 후처리 GBDT (400×63)	~25,000	통과	❌ -4.1
무수축 타겟 인코딩	엔티티 수	통과(로컬)	❌ -929

정직한 **홀드아웃 분할** 통과는 필요조건이지 충분조건이 아니다. 자유도가 크면 **연도별·폴드별 특수 구조**를 외우고, 그것은 다음 기간에 남지 않는다. 룩업이 살아남은 이유는 **EB 수축 + 관측량 게이트**가 실효 자유도를 극단적으로 낮췄기 때문이다.

🔗 **시너지** 자유도를 낮추는 장치들은 **중첩해서 쓸수록 좋다**: EB 수축 × 관측량 게이트 × 계수 수축(×0.8) × 교차폴드 적합 — 이 대회 룩업은 넷을 전부 썼다.

2부 · 모델링

2.1 GBDT 채택

직관 테이블 데이터에서 트리 부스팅이 강한 이유는 **축 정렬 분할**이 실제 데이터의 규칙 ("2스트라이크이고 주자가 있으면...")과 형태가 같기 때문이다. 결측·범주·비선형을 별도 처리 없이 다룬다.

라이브러리	강점	이 대회 역할
LightGBM	속도·대용량, leaf-wise	기준선 / 회귀 서브모델
CatBoost	순서형 부스팅, 범주 자동	주력 arm (20멤버 GPU)
XGBoost	정규화·안정성	3-way 보조

장점 적은 튜닝으로 강한 기준선. 스케일링 불필요. 해석 도구 풍부.

단점 외삽 불가 — 학습 범위 밖 값에 취약. 시간이 흐르며 분포가 밀리면 **분할점이 어긋난다**. 고카디널리티 범주에 과적합.

🔗 **시너지 3.2 시즌 상대화**(분포 밀림을 보정해 외삽 약점을 댄다) · **2.3 시드 배경**(거의 항상 소폭 개선) · **2.4 회귀 목적함수**(같은 GBDT로 다른 오차 구조를 만든다)

⚡ **안티시너지 GBDT 비중 축소** — 이 대회에서 **일관되게 손해**였다(10%→968, 14%→915, 기준 1032). GBDT는 앙상블의 중심으로 두고 다른 것을 더하는 방식이 맞다.

실측 최고점은 GBDT 25~45% 구간에 집중. 그리고 CatBoost의 **cat_features 지정이 -15.49**로 오히려 손해였다 — "권장 설정"을 믿지 말고 재라.

2.2 신경망 채택(1종) 기각(다종)

직관 신경망이 GBDT를 이겨서 가치가 있는 게 아니다. **틀리는 방식이 달라서** 가치가 있다. 블렌드에서 중요한 것은 성능이 아니라 **오차의 방향**이다(6.3).

장점 GBDT와 오차 구조가 달라 블렌드 이득이 크다. 연속값의 부드러운 함수를 잘 표현. 임베딩으로 고카디널리티 ID를 다룰 수 있다.

단점 튜닝 비용·시드 분산이 크고, **outer fold 일반화가 구조적으로 약하다**.

언제 GBDT 기준선이 선 뒤에, **블렌드 멤버로만** 단독 최고 성능을 노리고 쓰는 것은 대개 손해.

⚡ **안티시너지** — 이게 핵심이다 **신경망 종류를 늘리는 것은** 일관되게 손해였다. 이유는 명확하다 — **신경망끼리는 서로 상관이 높다**(이 대회 실측 NN↔NN 거리 0.005~0.018, 필요치 0.029). 즉 두 번째 신경망은 **첫 번째와 거의 같은 방향**을 가리켜 p 기여가 0이다.

실측 MLP 한 종 블렌딩 → +36.24(1000점 첫 돌파). ✅

+ResNet+Transformer → 1008 ❌ | +TabNet → 993 ❌ (기준 1032)

TabM 블렌딩 → 로컬 888.43, **LB 818.07 (-70.36)**. 사후 outer 검증에서 GBDT 단독보다 -28.29 ❌

2.3 시드 예측 배경 채택

직관 같은 모델을 다른 시드로 여러 번 학습해 **예측을 평균**한다(가중치 평균 아님). 각 모델의 우연한 실수가 서로 상쇄된다.

장점 거의 항상 소폭 개선. 구현 단순, 위험 없음.

단점 비용이 시드 수에 비례하고 **이득이 빠르게 포화**한다.

⚡ **안티시너지 이미 배경된 모델에 시드를 더 늘리기** — 이 대회에서 5-seed → 15-seed SWA 로 바꾸니 **LB 1032 → 1020**. 분산이 더 작아야 할 쪽이 더 낮았고, 이것이 노이즈 바닥 발견의 계기가 됐다.

🔗 **시너지 1.2 노이즈 바닥** — 시드 배경용으로 학습한 모델들을 그대로 노이즈 측정에 재활용할 수 있다.

2.4 목적함수 선택 — 분류 vs 회귀 채택

직관 이진 라벨이어도 **MSE** 로 학습할 수 있다. 지표가 Brier 라면 회귀가 지표와 직결된다. 그리고 분류 모델과 **다른 곳에서 틀리므로** 블렌드 멤버로 가치가 있다.

🔗 **시너지 6.1 2차형식** — 회귀 모델은 지표와 목적함수가 일치해 Gram 계산의 가정과 잘 맞는다.
2.1 GBDT — 같은 라이브러리·같은 피처로 **추가 학습 비용 없이** 다른 방향의 모델을 얻는다.

실측 GBDT분류 + GBDT회귀 + MLP회귀 3-way 가 구조적 개선 **+12.53**. 최종 채택 비율 **0.40 / 0.40 / 0.20**.

2.5 지식 종류 기각

직관 강한 교사의 **소프트 예측**을 학생의 타겟으로 쓴다. 교사가 배포 불가여도 지식 일부를 옮길 수 있다.

📌 **두 가지를 반드시 확인하라.**

- ① **이득이 정보인가 수축인가** — 5.4 로 분해하라. 이 대회에서 증류 이득 +85.2 중 **정보향이 -83.4**, 전량이 분산 수축이었다.
- ② **교사가 특권 피처를 쓰는가** — 이 대회 교사 3종이 **예측 대상 사건 자체의 실측값**을 쓰고 있었다. 배포 불가 + 규정 위반이었다(8.3).

⚡ **안티시너지 증류 + 특권 피처 교사** = 배포 불가능한 지식을 학생에게 옮기려는 시도. 학생이 그 신호를 흉내낼 수 없으면 이득은 0 이거나 잡음이다.

3부 · 피쳐 엔지니어링

3.1 as-of 누적 피쳐와 분해 채택

직관 "이 시점까지의 누적 성적"은 주최자가 주면 누출 걱정 없이 쓸 수 있는 **공짜 정보**다. 그런데 그대로 넣으면 트리는 **수준(level)**만 본다. 사람이 실제로 보는 것은 "**평소보다 지금 잘하고 있는가**" 같은 **차분**이다. 그 차분을 명시적으로 만들어 주는 것이 분해다.

구현

```
level      = asof_rate          # 장기 실력
recent_dev = prev5_rate - asof_rate # 최근 폼 편차 ← 트리가 만들기 어려운 것
volume     = log1p(asof_n)      # 관측량 = 신뢰도
volatility  = |prev1_rate - prev5_rate| # 변동성
```

장점 트리가 스스로 만들기 어려운 **차분·비율**을 제공. 관측량을 함께 주면 모델이 **신뢰도를 학습**한다.

단점 누적값은 시간이 갈수록 **수준이 밀린다** (이 대회: 관측량 중앙값 571 → 2942). 그대로 두면 분할점이 어긋난다.

🔗 **시너지 3.2 시즌 상대화**(밀린 문제를 정확히 해결) · **3.5 라벨 디코딩**(같은 누적 컬럼에서 라벨을 더 얻는다) · **4.2 엔티티 록업**(누적 컬럼의 관측량이 게이트로 재사용된다)

⚡ **안티시너지 EWMA·최근성 재가중과 겹쳐 쓰기** — 둘 다 "최근 폼"을 보므로 정보가 중복되고, EWMA는 사건 경계를 넘으면 리키지가 된다(3.2 경고 참조).

실측 as-of 분해피쳐 46개 → **LB +146.80**. 단일 변경으로 대회 전체 최대 이득. 4개 기간 홀드아웃 일관성을 확인하고 채택했다.

3.2 시즌 상대화 기각(이 대회)

직관 시대가 바뀌면 같은 숫자의 의미가 달라진다. 각 행을 **자기 시대의 평균**과 비교해 상대화한다.

```
# 시즌 평균은 학습 프레임에서만 계산해 표로 저장
# 표에 없는 시즌(테스트)은 마지막 학습 시즌 값으로 대체
# → 행 자신의 season 만 보는 lookup 이라 행 독립이 유지된다
val = row[col] - season_mean.get(row.season, last_train_mean)
```

실측 이 대회 실험에서는 **단조 하락** → **기각**. 그러나 팀원의 독립 파이프라인은 이 기법을 채택했고 그 모델이 **+7.56**을 냈다. → **기법 자체**가 아니라 **구현·맥락이 결과를 가른다**. 분포 밀림이 크면 다시 볼 것.

⚠️ 최근성 가중의 리키지

EWMA 형태로 "최근 경기 가중"을 주면 이득이 크게 나오는 경우가 있는데, 같은 사건 단위(경기·세션) 안의 **미래 정보**가 새어 들어간 것일 수 있다. 이 대회 실측: EWMA 최근성 이득 **+98점의 100%**가 경기내 리키지였다.

이득이 비정상적으로 크면 리키지부터 의심하라.

3.3 도메인 물리 파생 피처 기각

직관 도메인 지식으로 원시 측정값을 조합해 의미 있는 양을 만든다. 성공하면 남이 못 가진 정보가 된다. 그러나 대부분 실패한다 — 원본의 결정함수라 정보량이 0인 경우가 많기 때문이다.

만들기 전에 반드시 이 검사를 하라 (6초면 된다)

```
# 새 파생 피처가 기존 조인키의 결정함수인가?  
df.groupby(join_keys)[new_feature].std().max() # 0 이면 정보량 0 → 버린다
```

이 대회에서 트랙맨 파생 블록 전체가 조인키 7개의 결정함수였다(셀 내 sd=0).

⚡ 안티시너지 물리 파생 + 기존 집계 피처 — 집계에서 파생을 만들면 정보가 늘지 않고 차원만 늘는다. 이 대회 실측: 물리 137피처 블록 (+3.27)이 원본 43컬럼(+5.20)보다 낮았다 — 희석만 했다.

실측 터널링 3종 → +1.37 | 물리 9종 → -361.17 | 모델로 만들었을 때 잔차 설명력 $p = 0.00\%$

3.4 잠재변수 (CFA) 채택

직관 상관된 수많은 피처 뒤에 소수의 공통 요인이 있다고 가정하고 그것을 추정한다. PCA와 달리 CFA는 도메인 가설로 요인 구조를 미리 지정한다.

단점 GBDT 는 이미 상관 피처를 잘 다루므로 이득이 작다. 요인 적재는 **train 에서만** 적합해야 한다.

언제 피처가 수백 개고 강하게 상관될 때. 선형 모델·신경망에 특히 유용(GBDT 에는 덜하다).

🔗 시너지 2.2 신경망 — 차원이 줄고 잡음이 걸히면 신경망 학습이 안정된다. GBDT 보다 궁합이 낫다.

3.5 숨은 라벨 디코딩 채택 ★ 재사용 가치 최상

직관 누적 통계는 사건이 하나씩 더해진 결과다. 그러면 연속된 두 행의 차이는 그 사이에 일어난 한 사건의 결과다. 즉 주최자가 라벨 하나만 줘도, 누적 컬럼이 여러 개면 **라벨 차원을 여러 개** 복원할 수 있다.

형식

```
outcome(이번 사건) = rate(다음행) · n(다음행) - rate(이번행) · n(이번행)  
# 같은 엔티티의 연속 행이고 n 이 정확히 1 증가하는 쌍만 사용  
# 결과가 {0, 1} 에 충분히 가까운 행만 채택  
  
# ★ 반드시 검사: 이미 아는 라벨 차원으로 정확도를 확인한다
```

장점 없던 라벨이 생긴다. **train 안에서만** 하므로 규정 안전. 비용이 거의 0.

단점 연속 행이 확보돼야 한다. 반올림 처리 필요. 복원했다고 유용한 것은 아니다.

언제 데이터에 누적·이동평균 컬럼이 있으면 무조건.

🔗 시너지 — 이 대회 최고의 조합 4.3 대비 타겟 록업. 디코딩으로 얻은 보조 라벨을 록업의 타겟으로 쓴다. 디코딩 자체는 점수를 안 주지만, 록업에 공급되면 +11.00이 된다. → 디코딩은 단독 기법이 아니라 재료 공급 기법이다.

실측 5개 결과 차원 복원, 알려진 차원과 대조해 99.95% 일치, 복원률 99.8%. 그중 하나(reverse-middle 대비)만 록업 타겟으로 유효했고 나머지는 대조군 미만이었다.

3.6 상태 복원 (창 대수) 채택

직관 "시즌 누적"과 "직전 경기들 누적"이 둘 다 있으면, 그 차이가 지금 경기에서 진행된 분량이다. 아무도 안 주는 현재 진행 상태를 대수로 역산해 낸다.

```
j = 총누적_n - (시즌앵커_n + 직전경기들_n)      # 이번 사건 인덱스
s = 총누적_s - (시즌앵커_s + 직전경기들_s)      # 이번 진행 성공수
# 유일해가 안 나오면 train 적합 밴드 안에서 배수 탐색

correction = sigmoid(logit p + b·(s - j·p)/(j + K))  # 복원된 행에만
```

단점 대수가 복잡하고 커버리지가 제한적. 밴드 탐색은 정밀도 임계를 정해야 한다.

🔗 시너지 3.1 as-of 분해(같은 누적 컬럼을 쓴다) · 5.2 로짓 시프트(보정 형태가 로짓 가산이라 자연스럽게 연결)

실측 LB +17.09, 커버리지 확장으로 +6.06 추가.

4부 · 엔티티 룩업 — 이 대회 최대 성과

4.1 타겟 인코딩과 EB 수축 채택

직관 "이 투수는 평소 성공률이 얼마인가"를 한 숫자로 넣는다. 문제는 3번 던진 투수도 성공률이 계산된다는 것. 그래서 관측이 적으면 전체 평균 쪽으로 끌어당긴다(수축).

$$r_{\text{entity}} = (\text{성공수} + K \cdot r_{\text{parent}}) / (\text{관측수} + K) \quad \# K = 50 \sim 200$$

K 는 "가상의 관측 K 번을 부모 수준으로 미리 넣어둔다"는 뜻이다

📌 세 가지를 어기면 재앙이다

- ① 수축을 반드시 넣는다 — 무수축 버전이 이 대회에서 LB 103(정상 1032)을 냈다
- ② out-of-fold 로 만든다 — 자기 행의 라벨이 자기 인코딩에 들어가면 안 된다
- ③ 표는 train 에서만 — 홀드아웃 기간을 표 생성에 쓰면 계수가 폭주한다(실측 10배)

⚡ 안티시너지 타겟 인코딩 + 고자유도 모델 — 인코딩이 이미 라벨 정보를 압축해 넣었는데 그 위에 자유도 큰 모델을 얹으면 누출이 증폭된다. 인코딩을 쓸수록 모델은 보수적으로.

4.2 이중 중심화 잔차 룩업 채택 ★★ 최대 성과

직관 "투수 A는 잘한다"(엔티티 주효과)와 "2스트라이크에서는 다들 잘한다"(문맥 주효과)는 모델이 이미 안다. 남는 것은 "투수 A는 요 독 2스트라이크에서 잘한다"는 개인별 상호작용이다. 이것 뽑으려면 두 주효과를 모두 빼야 한다. 그래서 이중 중심화다.

형식

```
dev = r(엔티티, 셀, 제3축) - r(엔티티, 상위셀)      ← 엔티티 주효과 제거
      - [ 리그 r(셀, 제3축) - 리그 r(상위셀) ]      ← 문맥 주효과 제거

shift = gate(관측량 ≥ n_min) · (β1·dev_cell + β2·dev_coarse)
p ← sigmoid(logit p + shift)
```

0 과 +20 을 가르는 5요소

#	요소	빠지면 무슨 일이
1	이중 중심화	주효과에 묻혀 상호작용이 안 보인다
2	제3축 (매치업 성격의 축)	2축만으로는 정확히 0이 나온다
3	EB 수축 (부모=엔티티×상위셀)	희소 셀의 잡음이 신호를 덮는다
4	관측량 게이트	관측 적은 엔티티가 해를 끼친다
5	β 를 다른 폴드에서 적합 + ×0.8	폴드내 최적은 전이되지 않는다

장점 모델 재학습 없이 사후처리로 적용. 자유도가 극히 낮아 전이가 잘 된다(1.3). 완전한 행 독립.

단점 제3축 선택이 결정적인데 어떤 축이 먹힐지는 재 봐야 한다. 그리고 대부분의 축은 안 먹힌다(4.5).

언제 반복 등장하는 엔티티(선수·상점·사용자)와 문맥 변수가 있는 **모든 대회**. 베이스 모델 직후 최우선.

🔗 시너지 3.1 as-of(관측량 컬럼이 게이트로 직결) · 3.5 디코딩(대비 타겟 공급) · 5.2 로짓 시프트(적용 형태가 같아 체인으로 이어진다) · 7.5 무작위 대조군(축 선택 시 필수)

⚡ 안티시너지 같은 계열 룩업을 여러 개 쌓기 — 라벨 조건부 표들은 서로 상관이 0.90 이상이라 두 번째부터 이득이 급감한다. 다른 타겟으로 가야 한다(4.3).

실측 정직 측정 +19.70 → LB +15.21. 이후 평할 강도 조정으로 +1.56 추가.

4.3 대비 타겟 룩업 채택

직관 같은 표 구조에 타겟만 바꾼다. 원 라벨이 아니라 "실패했을 때 어느 쪽으로 실패했는가"의 대비를 타겟으로 삼으면, 원 라벨 룩업이 못 보는 다른 축의 개인차가 잡힌다.

🔗 시너지 3.5 디코딩 없이는 불가능하다 — 보조 라벨이 있어야 대비를 만들 수 있다. 이 둘은 **사실상 한 세트**다.

실측 +11.00 → LB +5.47. 같은 카-같은 레시피, 타겟만 교체했는데 원 룩업과 거의 겹치지 않는 이득이 나왔다.

4.4 축 스위치 (무작위 대조군 포함) 채택

직관 제3축이 무엇이어야 하는지는 **이론으로 못 정한다**. 여러 개를 훑어야 한다. 그런데 여러 개를 훑으면 **우연히 좋아 보이는 것**이 반드시 나온다. 그래서 **가짜 축**을 같이 넣어 "우연의 크기"를 함께 잔다.

🔗 시너지 7.5 무작위 대조군과 분리 불가능한 한 세트.

실측 11개 실후보 최고 +0.49 vs 무작위 최고 +0.94 → 즉시 기각. 대조군이 없었으면 "발견"으로 보고됐을 것이다.

4.5 실패한 변형들 기각

변형	결과	해석
제3축을 문맥 변수로 교체 (11종)	최고 +0.49 < 대조군 +0.94	문맥은 엔티티와 안정적 상호작용을 만들지 않는다
대칭 엔티티로 같은 표 (상대편 기준)	누출 오라클조차 +0.06	타겟을 한쪽 엔티티가 통제 하면 비대칭이 실재한다
타겟을 다른 차원으로 (7종)	최고 +1.89 < 대조군 +2.44	차원을 늘린다고 되는 게 아니다. 하나만 먹혔다
게이트·평할 강도 격자 (5×4)	출하값이 이미 최적, 여지 +0.00	교차폴드로 고른 값은 대개 이미 최적이다

다른 대회에서는 대칭 엔티티는 **양쪽이 대등하게 결과를 통제하는 문제**(예: 대전 게임 승패)라면 유효할 수 있다. "실패"는 이 데이터에서의 실패다.

5부 · 캘리브레이션과 사후보정

5.1 아핀 보정 채택

직관 모델이 "전반적으로 소심하다"거나 "전체적으로 높게 본다"는 **계통 오차**는 예측을 늘리고 옮기는 것만으로 고칠 수 있다. 파라미터가 2개뿐이라 과적합이 거의 없다.

```
p' = clip(0.5 + SCALE·(p - 0.5) + SHIFT, eps, 1-eps)
# SCALE > 1 : 더 확신 있게(분산 확대)      SCALE < 1 : 수축
```

🔗 시너지 — 강력하다 5.3 LB 역산. 파라미터가 2개뿐이라 **제출 2~3회로 최적값을 직접 찾을 수 있다**. 자유도가 낮은 기법만이 LB 최적화를 감당한다.

⚡ 안티시너지 SCALE 과 SHIFT 를 동시에 바꿔 제출하기 — 델타 해석이 불가능해진다. 이 대회에서 실제로 두 개를 묶어 냈다가 -3.39 의 원인을 특정하지 못했다.

실측 SCALE 1.10 + SHIFT -0.00452 → **+12.90**
SCALE 1.15 로 확대 → **-7.41** | SHIFT -0.01 로 확대 → **-27.35**
→ **좁은 국소 최적점**. 양방향 외삽 모두 실패. 이 축은 조기에 닫았다.

5.2 로짓 셀 오프셋 채택

직관 확률 공간에서 더하면 0/1 근처에서 범위를 벗어난다. **로짓에서 더하면** 자연스럽다. 그리고 문맥 셀마다 다른 값을 주되, **전역 스케일 U 하나로 강도를 묶어** 자유도를 통제한다.

```
z = log(p/(1-p)); z[cell == k] += U · offset[k]; p = sigmoid(z)
# 셀이 12개여도 실질 자유도는 U 하나에 가깝다 — 이것이 핵심 설계다
```

🔗 시너지 1.3 자유도 통제의 모범 사례. 3.6 상태 복원·4.2 룩업과 같은 로짓 가산 형태라 **체인으로 자연스럽게 이어진다**.

실측 U=0.35 채택. U 를 0.40/0.50 으로 올리는 시도는 각각 **-0.47 / -1.69** → 축 종결.

5.3 LB 역산 — 리더보드를 측정 도구로 채택

직관 로컬 검증이 못 잡는 것(시간 외삽 효과)을 **리더보드가 직접 측정해 준다**. 계수 하나만 바꿔 3번 제출하면 **포물선 세 점이 되고**, 꼭짓점이 최적값이다.

```
points = [(1.2, 1076.19), (1.05, 1083.32), (1.5, 1078.86)]
a,b,c = np.polyfit(*zip(*points), 2); x* = -b/(2a)
```

언제 **최고점 기준 대회**(나쁜 제출이 기록을 안 깎음)면 하방이 0이라 거의 공짜다. 슬롯만 소모된다.

🔗 시너지 5.1 아핀·5.2 셀 오프셋 — 파라미터가 적은 기법에만 쓸 수 있다.

⚡ 안티시너지 여러 개를 동시에 바꾸기 — 한 번에 하나만 바꿔야 델타가 그 축의 순수 신호가 된다.

실측 로컬로는 못 찾았을 최적값을 특정, 그 계수로 +17.09 획득에 기여.

5.4 분산 수축 분해 채택 ★ 항상 함께 보고할 것

직관 Brier 는 확신을 줄이기만 해도 좋아진다. 그래서 "개선"의 상당수가 정보가 늘어난 게 아니라 그냥 조심해진 것이다. 이 둘은 반드시 분리해야 한다. 왜냐하면 수축은 이미 아핀 보정이 하고 있어서 중복이기 때문이다.

```
# 후보의 이득에서 '순수 수축'으로 설명되는 몫을 뺀다
s*      = argmax skill(0.5 + s*(p-0.5))      # 다른 폴드에서 적합
shrink_d = (0.5 + s*(p-0.5)) - p
info_d   = cand_d - proj(cand_d, shrink_d)    # 수축 성분 제거
정보 이득 = skill(p + info_d) - skill(p)
```

🔗 시너지 2.5 종류 판정의 필수 도구. 모든 새 후보의 이득 보고에 함께 붙여라.

실측 종류 축의 이득 +85.2 중 정보항 -83.4 → 전량이 수축임이 드러나 종결.

반대로 잔차 후처리 후보는 수축 기여가 -2%로 진짜 정보였는데도 다른 이유로 실패했다 (5.5) — 수축 검사만으로는 부족하다는 뜻이다.

5.5 고자유도 사후처리의 함정 기각 ★ 이 대회 최대 교훈

직관 "모델이 남긴 잔차를 다른 모델로 예측해 더하면 되지 않나?" — 자연스러운 발상이고, 정직한 홀드아웃 분할도 통과한다. 그런데 실전에서 죽는다. 이유는 그 모델이 "이 데이터의 오차"가 아니라 "이 기간의 오차"를 외우기 때문이다.

실제로 일어난 일 — 이 함정을 피하는 법을 여기서 배우는 게 낫다.

① 완전 분리 3분할(학습 / 계수적합 / 평가)에서 +6.65 획득 → 출하

② 실전 -4.1

③ 사후 진단에서 세 가지가 전부 기각했고 전부 출하 전에 볼 수 있었다:

학습	계수	평가	β	이득
기간1	기간2	기간3	+0.070	-22.35
기간2	기간3	기간4	-0.041	+4.88
기간1+2	기간3	기간4	-0.140	+3.21

㉓ 다른 시점에서 재현 안 됨 ㉔ 계수 부호가 폴드마다 뒤집힘 ㉕ 시드 하나로 +6.65 → +3.21 (효과가 시드 잡음 안)

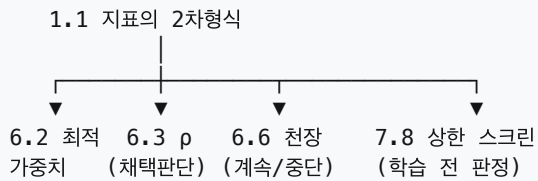
⚡ 안티시너지 — 일반 법칙 고자유도 기법 + 시간 외삽은 구조적으로 위험하다. 스테킹·Isotonic·잔차 모델·무수축 인코딩이 전부 여기 해당한다. 시간 축이 없는 대회(랜덤 분할)라면 같은 기법이 안전할 수 있다.

그래도 쓰려면 ① 최소 5시드 ② 양방향 폴드에서 계수 부호 일치 확인 ③ 출하할 바로 그 산출물을 검증 (창을 밀거나 재학습했으면 그건 새 후보다)

6부 · 앙상블

6.1 2차형식 — 6.1~6.6의 공통 토대

1.1에서 유도한 Gram 행렬 M 하나가 모든 조합의 점수를 결정한다. 아래 절들은 전부 이 행렬의 활용이다.



6.2 최적 가중치 채택

$w^* \propto M^{-1} \cdot 1$ # 아핀(음수 허용)
심플렉스($w \geq 0$)면 부분집합 전수 탐색이 가장 안전 (모델 수가 적을 때)

⚠ 가중치를 평가 폴드에서 적합하면 반드시 과적합한다. 이 대회 실측 — 모델 57종 동시 최적화: 적합 폴드 +148.89 / 평가 폴드 -15.94(약한 정규화). 정규화를 강하게 걸어야 평가 폴드에 +2.73이 남았고, 그것이 진짜 천장이었다.

6.3 ρ — 이득을 결정하는 유일한 값 채택 ★★

직관 새 모델이 블렌드에 보태는 값은 그 모델이 얼마나 좋은가도, 얼마나 다른가도 아니다. "기존 조합이 아직 못 맞히고 있는 부분을, 그 모델의 고유한 방향이 얼마나 설명하는가" 하나다.

$\Delta = (C - s_base) \cdot \rho^2$
 $\rho = \text{corr}(\text{현재 블렌드의 잔차}, \text{새 모델의 고유방향})$
고유방향 = 새 모델의 오차를 기존 모델들 오차의 아핀 부분공간에 투영하고 남은 성분

이 두 가지가 이 책에서 가장 반직관적이고, 둘 다 실측으로 확인됐다.

① 다양성의 "크기"는 레버가 아니다.

고유방향을 3배로 키워 상관을 0.905 → 0.583으로 낮춰도 기여는 완전히 불변이고 가중치만 정확히 1/m로 준다. 블렌드가 알아서 재조정하기 때문이다.

→ "상관을 낮추자"를 목표로 삼으면 노이즈 주입으로 '달성'되면서 점수가 깎인다. 실측: 노이즈 주입 → 상관 0.905 → 0.838, 기여는 하락.

② 단독 성능도 레버가 아니다.

성능을 올리면 예측이 기존 모델들의 합의 쪽으로 끌려가 다양성이 그만큼 깎인다. 이 대회 변형 11종에서 성능 ↔ 거리 상관 -0.980, 단독 성능 최고 모델이 최적 가중치 0.000을 받았다.

🔗 시너지 7.8 상한 스크린 — ρ 의 상한을 모델을 학습하기 전에 짚 수 있다. 이 둘이 한 세트.

실측 물리 피쳐 모델의 $\rho = 0.66\%$, 게이트 통과 필요치 1.10% → 튜닝으로는 도달 불가능이 계산으로 증명돼 축을 단았다.

6.4 게이팅 블렌드 채택

직관 어떤 모델이 특정 구간에서만 구조적으로 약하다면(그 구간의 피처를 안 썼다든지), 그 구간의 가중치만 낮추면 된다. 행 자신의 컬럼으로 정하므로 규정도 안전하다.

```
w = np.where(row['segment'] == '약한구간', 0.20, 0.55)    # 행 자신의 컬럼만  
p = w * A + (1 - w) * B
```

🔗 시너지 7.2 레짐 점검 — 어느 구간이 약한지는 레짐 분석이 알려준다. 진단 → 처방의 관계.

실측 한 모델이 특정 리그 구분을 피처에서 제외해 그 행에 미보정이었다 → 그 행만 가중치 0.55 → 0.20 으로 낮춰 채택.

6.5 스택킹 기각

단점 자유도가 크고 OOF 구성이 조금만 틀려도 누출된다. 시간 외삽에서 특히 약하다(5.5의 일반 법칙).

실측 여러 형태를 시도했으나 단순 가중 블렌드를 넘지 못했다. 최종 채택은 전부 선형 블렌드다.

6.6 라이브러리 천장 채택

직관 "지금 가진 모델을 전부, 최적으로 조합하면 어디까지 가능한가?" 이 값이 목표에 한참 못 미치면 조합을 더 뒤지는 것은 무의미하고 새 정보원이 필요하다. 캠페인을 계속할지 접을지를 결정하는 도구다.

```
# 가중치는 fit 폴드에서만 적합, eval 폴드에서 실현치를 본다  
# 정규화 강도를 반드시 훑을 것 - 약하면 과적합이 그대로 드러난다
```

⚠️ 계산 전에 특권 피처 모델을 제외하라(8.3). 이 대회에서 교사 모델을 포함해 계산한 천장이 +113(도달 가능)이었는데, 빼고 다시 계산하니 +2.1이었다. 하마터면 캠페인 전체를 잘못된 방향으로 끌 뻘했다.

7부 · 검증 방법론

가장 비싸게 배운 부분이다. 여기의 규율 하나를 어겨 실전 점수를 잃었다.

7.1 폴드 설계와 예측 시계 채택

직관 시간 데이터는 **미래로만** 검증해야 한다. 그런데 그것만으로는 부족하다 — “몇 년 뒤를 예측하는가”(예측 시계)가 폴드와 배포에서 같아야 한다.

예측 시계 합정 — 실측

어떤 모델이 1년 격차 폴드에서 기여 $+33.80$, 2년 격차에서 $+0.00$ (폴드내 오라클조차 가중치 0)이었다. 같은 모델, 같은 데이터인데 격차만 달랐다.

새 모델을 평가할 때 “학습에 쓴 마지막 시점”과 “평가 시점”의 간격을 먼저 적어라.

7.2 레짐 점검 채택

직관 데이터의 일부 세그먼트가 어느 시점에 성질이 바뀌면 그 구간을 포함한 폴드는 못 믿는다. 폴드를 쓰기 전에 세그먼트별 라벨률·상관의 시간 추이를 그려라.

실측 한 세그먼트가 특정 연도에 상관 $0.709 \rightarrow 0.473$ 으로 붕괴 $\rightarrow$ 그 폴드는 해당 세그먼트를 제외하고 채점했다. 그 결과 그 폴드가 LB 최근사 폴드가 됐다(평균절대오차 3.03 vs 다른 폴드 13.72).

주의 세그먼트를 제외하면 그 세그먼트에만 작용하는 파라미터는 그 폴드로 판정할 수 없다. 적용 범위를 항상 확인하라.

7.3 중첩 검증 채택

튜닝은 inner 에서만, 최종 검증은 outer 에서 한 번만. 실측: inner 노이즈 ± 11.98 / outer ± 36.87 — outer 가 3배다. inner 에서 잡은 개선의 상당수가 outer 에서 사라진다.

7.4 사전 등록 채택

직관 결과를 본 뒤에 기준을 정하면 반드시 기준이 결과에 맞춰진다. 결과를 보기 전에 로그에 적는 것만으로 자기기만의 대부분이 막힌다.

실측 이 규율을 지킨 실험은 전부 정직한 결론이 나왔고, 어긴 실험이 실전 -4.1 을 냈다.

7.5 무작위 대조군 채택 ★★

직관 후보 2,000개를 훑으면 라벨과 아무 관계없는 것도 몇 개는 좋아 보인다. 그 “우연의 크기”를 이론으로 계산하는 대신 가짜 후보를 같이 넣어 실측한다. 파이프라인의 실제 자유도를 반영하므로 이론적 보정보다 정확하다.

```
for i in range(N_RANDOM):
    candidates[f'RANDOM:{i}'] = rng.standard_normal(n)    # 또는 무작위 그룹 분할
threshold = max(사전기준, random_scores.max())
```

언제 후보를 10개 이상 훑는 모든 탐색.

🔗 시너지 4.4 축 스위치 9.1 대량 탐색과 분리 불가능.

실측 두 번의 스위치에서 실후보 최고가 무작위 최고보다 낮았다(+0.49 vs +0.94, +1.89 vs +2.44). 대조군이 없었으면 둘 다 "발견"으로 보고됐을 것이다. 누적 4,833 후보 탐색에서 통과 0.

7.6 시드 규율 채택

최소 5시드. 2~3시드는 노이즈를 4~5배 과소평가한다.

이 대회에서 1폴드 × 1시드로 판정하고 출하한 것이 실전 -4.1의 직접 원인이다. 사후에 시드만 바꾸니 +6.65 → +3.21로 움직였다.

페어드 부트스트랩: 같은 행에 대한 두 예측의 차이를 부트스트랩한다. 독립 부트스트랩보다 구간이 훨씬 좁아 작은 차이를 판정할 수 있다.

7.7 전이비 채택

변경 유형	로컬→실전 전이비
다른 파이프라인 (정직 홀드아웃)	~1.0
엔티티 잔차 록업	0.76
블렌드 수준 사후처리	~0.52
모델 내부 가중치·캘리브레이션	~0.16 (순위 불일치)

축마다 다르다. 상수 하나로 환산하면 틀린다. 제출할 때마다 (로컬Δ, 실전Δ) 쌍을 기록하라.

7.8 상한 스크린 채택 ★★ 학습 전에 천장을 켜다

직관 "이 정보원으로 모델을 만들면 도움이 될까?"를 모델을 만들지 않고 답한다. 정보원으로 현재 잔차를 직접 회귀해 보면, 그 정보원 위에 세운 어떤 모델도 그 값을 넘지 못한다. 즉 상한이 나온다.

```
# 3분할 중첩으로 누출 차단
#   폴드A+B 학습 → 폴드C 에서 계수 적합 → 폴드D 에서 평가
gain = skill(chain + β·model.predict(X[D])) - skill(chain)
rho  = sqrt(gain / (C - base_skill))
```

언제 새 모델·새 피쳐 블록 제안을 받으면 가장 먼저. GPU 를 켜기 전에.

🔗 시너지 6.3 ρ와 한 세트 — 필요 ρ 를 6.3 으로 계산하고, 가용 ρ 를 7.8 로 측정해 비교한다.

실측 물리 77피처 → ρ 0.00% | 전체 137피처 → 0.57% | 원본 43컬럼 → 0.72% | 필요치 1.10%
→ 그 방향은 원리상 불가능. 먼저 돌렸으면 GPU 시간을 아꼈다.

8부 · 규정 준수와 누출 탐지

8.1 행 독립성 검증 채택 ★★

직관 "각 행을 독립적으로 예측하라"는 규정을 지켰는지 코드를 읽어서 확인하려면 전체를 다 읽어야 하고 놓치기 쉽다. 대신 **같은 행을 다른 프레임에 담아 넣어보면** 값이 달라지는지로 알 수 있다. **행동 검증이 정적 리뷰보다 강하다.**

검사	잡는 것	이 대회 실측
SHUFFLE 불변성	행 순서 의존 (rolling / shift / 정렬)	0.0e+00
SUBSET 불변성	프레임 통계 의존 (groupby / 평균 / 분위수)	4.7e-10
SOLO (1행씩)	위 전부 — 가장 강한 검사	7.9e-09

진짜 누출은 $1e-3$ 안팎, 부동소수점 잡음은 $1e-9$ 수준이라 명확히 갈린다.

언제 대회 1일차. 나중에 붙이면 이미 오염된 것을 못 본다.

주의 1행 프레임은 CSV dtype 추론이 달라질 수 있다. 실측: 값이 전부 숫자인 문자열 카테고리 '123' 이 1행 프레임에서 int64 로 파싱돼 인코더가 크래시했다. **모델 결함이 아니다.**

8.2 누출의 유형

유형	증상	탐지
테스트 프레임 통계 사용	부분집합 결과가 다름	SUBSET 검사
행 순서 의존	셔플 결과가 다름	SHUFFLE 검사
타겟 인코딩 자기누출	로컬 폭등, 실전 폭락	OOF 구성 확인
표 생성에 홀드아웃 포함	계수가 정상의 10배	계수 크기 확인
사건 경계 내 미래 정보	이득이 비정상적으로 큼	경계 넘는지 확인
특권 피쳐	단독 성능이 비정상적으로 높음	8.3

실측 — 계수 폭주 신호 표가 이미 본 폴드에서 계수를 적합하니 **정상 0.51 → 4.97**이 되고 정직한 폴드에서 **-245**가 났다. **계수 크기가 예상의 몇 배면 누출을 의심하라.**

8.3 특권 피쳐 판별 채택 ★

판별 원칙: 어떤 모델의 단독 성능이 주력보다 크게 높으면 그 자체가 경고다. 진짜라면 이미 쓰이고 있었을 것이다.

확인 절차 (4단계)

- ① 그 모델의 피쳐 목록·메타데이터를 연다
- ② 참조하는 외부 파일을 연다
- ③ 그 파일의 행 수가 학습 데이터와 1:1이면 = 각 행의 개별 값 → **특권 피쳐**
- ④ 테스트 데이터에 그 컬럼이 있는지 — 없으면 배포 자체가 불가능

실측 교사 3종이 예측 대상 사건 자체의 실측값을 쓰고 있었고, 그 파일은 학습 데이터와 정확히 1:1이었다. 포함해 계산한 천장 +113(도달 가능) → 제외하니 +2.1.

9부 · 시너지 지도

9.1 시너지 매트릭스

행이 세로축 기법, 열이 가로축 기법. ++ 강한 상승, + 보완, · 무관, - 중복, -- 위험.

\	asof 분해	디코딩	엔티티 록업	대비 타겟	아핀 보정	셀 오프셋	게이팅	신경망 추가	고자유도 사후처리	대조군
as-of 분해	—	++	++	+	·	·	·	+	—	·
라벨 디코딩	++	—	+	++	·	·	·	·	·	·
엔티티 록업	++	+	—	++	+	++	·	·	--	++
대비 타겟	+	++	++	—	·	+	·	·	·	+
아핀 보정	·	·	+	·	—	—	·	·	—	·
셀 오프셋	·	·	++	+	—	—	+	·	·	·
게이팅 블렌드	·	·	·	·	·	+	—	+	·	·
신경망 종류 추가	+	·	·	·	·	·	+	--	·	·
고자유도 사후처리	—	·	--	·	—	·	·	·	—	+
무작위 대조군	·	·	++	+	·	·	·	·	+	—

9.2 핵심 결합 5종 — 이것만 외워도 된다

① 디코딩 → 대비 타겟 → 엔티티 록업 (실측 +11.00)

누적 컬럼 —(차분)—→ 보조 라벨 —(대비)—→ 록업 타겟 → 로짓 시프트

디코딩은 단독으로는 점수를 주지 않는다. 록업에 재료를 공급할 때 비로소 이득이 된다. 기법을 단독 성능으로 평가하면 이 조합을 놓친다.

② as-of 분해 + 엔티티 록업 (합계 +170)

둘 다 엔티티 이력을 쓰지만 **층이 다르다** — 분해는 **모델 입력**으로, 록업은 **모델 출력의 사후 보정**으로 들어간다. 그래서 겹치지 않는다. 게다가 분해가 만드는 **관측량 컬럼**이 록업의 **게이트**로 그대로 재사용된다.

③ 아핀 보정 + LB 역산 (실측 +12.9, 그리고 +17.09 기여)

파라미터가 **2개뿐이라** 제출 3회로 최적값을 직접 찾을 수 있다. **자유도가 낮은 기법만이 LB 최적화를 감당한다** — 이것이 이 조합의 조건이다. 자유도가 크면 LB 3점으로는 아무것도 못 정한다.

④ 축 스위치 + 무작위 대조군 (잘못된 채택 2회 방지)

제3축은 이론으로 못 정하니 **훑어야** 하는데, **훑으면 우연이 나온다**. 둘은 **분리 불가능한 한 세트**다. 대조군 없는 스위치는 자기기만 장치다.

⑤ ρ 계산 + 상한 스크린 (GPU 시간 절약)

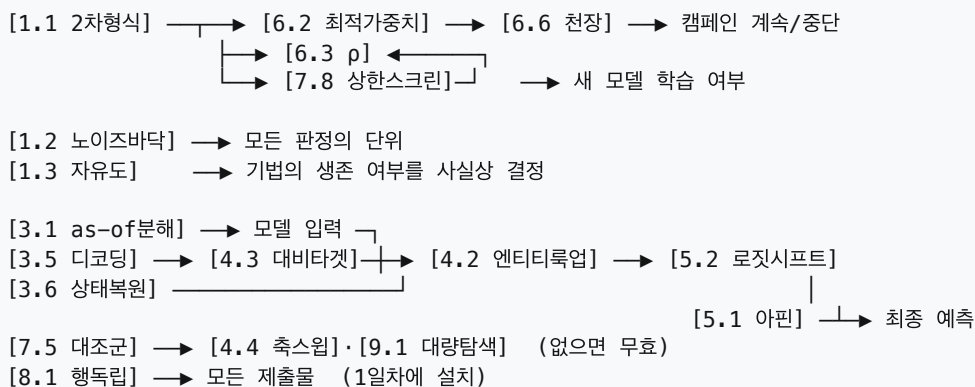
6.3 $\rho \rightarrow$ "이 목표에 필요한 ρ 는 1.10%"
 7.8 스크린 $\rightarrow$ "이 정보원의 가용 ρ 는 0.00%"
 ▼
 학습하지 않는다

필요치와 가용치를 같은 단위로 비교한다. 이 대회에서 이 조합이 물리 피쳐 축을 종결시켰다.

9.3 금지 조합 7종

조합	왜 위험한가	실측
고자유도 사후처리 × 시간 외삽	"이 데이터의 오차"가 아니라 "이 기간의 오차"를 외운다. 정직한 분할도 통과한다	-4.1
타겟 인코딩 × 수축 없음	희소 엔티티의 라벨이 그대로 새어 들어간다	-929
신경망 × 신경망	서로 상관이 높아(거리 0.005~0.018 < 필요 0.029) 두 번째의 ρ 기여가 0	-24~-39
같은 계열 룩업 여러 개	표들끼리 상관 > 0.90 $\rightarrow$ 두 번째부터 이득 급감. 타겟을 바꿔야 한다	—
물리 파생 × 기존 집계	집계의 결정함수라 정보가 안 들고 차원만 늘어 희석된다	137피쳐가 43피쳐보다 낮음
종류 × 특권 피쳐 교사	학생이 흉내낼 수 없는 신호. 이득은 수축이거나 잡음	정보량 -83.4
GPU × 상한 미측정	연산이 빨라지면 틀린 방향으로도 빨리 간다. GPU 는 탐색 확대가 아니라 검증된 방향의 실행 도구다	T4 세션 다수 낭비
제출 한 번에 두 개 변경	델타 해석 불가 $\rightarrow$ 슬롯을 쓰고도 아무것도 배우지 못한다	원인 특정 실패

9.4 기법 의존 그래프



10부 · 운영

10.1 제출물 엔지니어링

#	검사	없어서 잃은 것
1	zip 위생 (캐시·학습데이터 동봉 금지)	용량·정보 유출
2	진입점이 최상위에 있는가	실행 실패
3	모든 <code>.py</code> 구문 검사	슬롯 소실
4	새로 푼 폴더에서 필수 데이터만 두고 실행	모듈 누락
5	출력 행수·범위·NaN·id 중복	0점
6	행 정렬 — id 로 reindex 했는가	100% 오정렬
7	두 번 실행해 바이트 동일	재현 불가
8	실행 시간·용량 여유	타임아웃
9	의존성 목록을 함부로 바꾸지 않기	설치 오류로 전체 실패

6번이 특히 위험하다. 여러 모델을 서브프로세스로 돌려 합치는 구조에서 각 모델의 출력 순서가 다를 수 있다.

`.reindex(sub[id])` 없이 `.values` 로 대입하면 전 행이 어긋난다(실측 최대오차 0.5132).

9번 실측: 새 모듈의 로컬 핀을 최상위 의존성 목록에 합치면서 한 줄이 잘못 들어갔고, 채점 서버에서 설치 오류로 제출이 통째로 실패했다. 채점 이력이 있는 목록만 쓰고, 일부는 의도적으로 버전을 고정하지 않는다.

10.2 GPU·클라우드 연산

직관 GPU 는 "할 수 있는 것"을 늘리지 않는다. "같은 것을 빨리" 할 뿐이다. 그런데 방향이 틀렸으면 빨리 가는 것이 손해다. 이 대회에서 GPU 투자 3건 중 2건이 전량 낭비였고, 그 2건은 몇 분짜리 계산으로 사전에 걸러낼 수 있었다.

10.2.1 GPU 를 켜기 전에 — 유일하게 중요한 질문

🔗 시너지 — 7.8 상한 스크린 "이 정보원으로 모델을 만들면 도움이 되는가"는 모델을 만들지 않고 답할 수 있다. 정보원으로 현재 잔차를 직접 회귀하면 그 위에 세운 어떤 모델도 넘지 못하는 상한이 나온다. 이 값이 필요치에 못 미치면 GPU 를 켜지 마라.

[7.8 상한 스크린] → 가용 $p = 0.00\%$
[6.3 필요 p] → 필요 $p = 1.10\%$

▼
GPU 를 켜지 않는다 ← 몇 분이면 나오는 판단

실측 물리 피쳐 arm 을 Colab T4 에서 변형 11종 학습했다. 결과는 전부 $p \approx 0$. 나중에 상한 스크린을 돌리니 그 블록의 잔차 설명력이 0.00% — 즉 어떤 모델을 어떻게 학습해도 불가능했고, 그 사실이 몇 분이면 나왔다.

10.2.2 🚩 GPU ≠ CPU — 판정과 배포는 같은 장치로

같은 하이퍼파라미터·같은 데이터인데 다른 모델이 나온다(실측).

비교	값
CatBoost GPU vs CPU AUC 차	+0.02
두 예측의 상관	0.939

합의 CPU 로 후보를 판정하고 GPU 로 배포하면 "판정한 것과 다른 물건"을 내는 것이다. GPU 구현은 히스토그램 분할·부동소수점 누적 순서가 다르고, 일부 파라미터의 동작(범주형 처리 등)도 CPU 와 다르다.

10.2.3 클라우드 GPU 운영 (실전 SOP)

pod 6개를 운영하며 확정된 두 규칙

- ① Secure 등급만 쓴다. 저가(Community) 등급은 4회 중 3회 GPU 불량이었다 — `nvidia-smi` 는 정상으로 보이는데 `torch.cuda.is_available()` 이 False. 우회 3종 전부 실패. Secure 는 4/4 정상.
- ② 중지(stop)하지 말고 종료(terminate)한다. 중지는 GPU 슬롯 반납이라 재시작이 보장되지 않는다(2회 확인). 산출물은 생성 즉시 회수한다.

```
# ① GPU 실물 확인 - False 면 즉시 종료하고 재생성 (이 한 줄이 시간을 아낀다)
ssh $H 'python3 -c "import torch;print(torch.cuda.is_available())"'
# ② 데이터·코드 전송 (로그·캐시 제외) ③ 의존성은 채점 이력이 있는 목록 그대로
# ④ 피쳐 캐시 생성 → 학습 → 산출물 즉시 회수
```

스테이징 약 10분. 전체 명령 시퀀스는 07_GPU_COMPUTE.md §2.3.

10.2.4 노트북 GPU(Colab 등) 워크플로

두 벌 규칙 — 이걸 어기면 측정 자체가 불가능해진다

- (a) 판정용 : 홀드아웃 기간을 제외하고 학습 → 그 기간 예측 .npy
- (b) 배포용 : 전 기간 학습 → 제출물에 탑재

실측: 배포용만 받았더니 홀드아웃 기간이 in-sample 이라 단독 성능이 1982(정직 측정 730.9)로 부풀었고, 그 모델의 가치를 잴 수 없었다.

실무 · 데이터 업로드가 병목이다 → 필요한 컬럼만 슬림 CSV(이 대회 368MB → 33.5MB)

- 세션이 끊기면 산출물이 사라진다 → 학습 직후 즉시 다운로드
- 무료 등급 GPU 로도 부스팅 20시드는 충분히 돌아간다

10.2.5 GPU 투자 결산 — 이 대회

대상	투입	회수
부스팅 20멤버 (주력 arm)	클라우드 A40, pod 6개	✅ 블렌드 한 축
물리 피쳐 arm 변형 11종	노트북 GPU 다수 세션	❌ ρ 0.00%
대체 구성 번들	노트북 GPU	❌ AUC 0.4985

⚡ 안티서너지 — 일반 법칙 GPU × 상한 미측정. 연산이 빨라지면 틀린 방향으로도 빨리 간다. GPU 는 "탐색 범위를 넓히는" 도구가 아니라 "이미 옳다고 판정된 방향을 빨리 실행하는" 도구로 써야 한다.

10.3 로컬 실행 인프라

증상	원인 / 대처
로그는 쌓이는데 결과가 없음	no-op 스크립트 . 로그에 숫자가 없으면 아무것도 안 한 것
프로세스가 CPU 0.0%로 정지	스레드 라이브러리 데드락. 스레드 수를 1로 강제. 진단: 느린 게 아니라 죽은 것이다
두 번째 학습부터 죽음	한 프로세스에서 반복 학습 불가 → 학습 1회 = 프로세스 1개
판정과 배포 결과가 다름	GPU ≠ CPU (10.2.2)
딥러닝 라이브러리 동시 로드 시 세그폴트	import 순서 조정 또는 프로세스 분리
grep 이 조용히 아무것도 안 함	셸이 와일드카드를 먼저 확장 → <code>find -exec</code>

```
export OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1 KMP_DUPLICATE_LIB_OK=TRUE
```

10.4 다중 에이전트 운용

1. 실행자와 판정자를 분리한다. 자기 결과를 자기가 판정하면 낙관 편향이 걸린다.
2. SSOT 파일 하나에 모든 판정을 append — 중복 실험 방지.
3. 판정 게이트를 코드로 고정한다.
4. 산출물 접수 규격: 새 모델은 두 벌(판정용 홀드아웃 예측 + 배포용). 홀드아웃 점수가 없는 산출물은 접수하지 않는다.
5. 에이전트 리포트는 기본 미검증. 직접 재현한 수치만 인용.
6. 조작이 발견되면 격리 + 라벨. 삭제하지 않는다(지우면 같은 주장이 돌아온다).

❗ 조작 리포트 판별 (실제 111건 격리)

- 존재하지 않는 채점 결과를 소수점까지 인용 · 빈 f-string(숫자를 계산하지 않고 적었다는 증거)
- 재현 불가능한 수치 · 이모지·"공식"·"검증 완료" 같은 과잉 수사

11부 · 실행 순서

11.1 대회 진행 순서

단계	할 일	왜 이 순서인가
0	규정에서 금지·허용 항목 특정	허용을 늦게 알면 손해다
1	행 독립성 하네스 설치	나중에 붙이면 이미 오염된 것을 못 본다
2	GBDT 기준선 + 5시드	신경망은 여기가 선 뒤에
3	노이즈 바닥 측정	이후 모든 판정의 단위가 된다
4	폴드 설계 + 레짐 점검	예측 시계를 배포와 맞춘다
5	as-of 분해	이 대회 최대 이득 (+146.8)
6	엔티티 잔차 록업	두 번째 이득 (+23.3). 5요소 준수
7	디코딩 + 상한 스크린	새 정보원 발굴, 학습 전 판정
8	앙상블 (ρ 로 채택 판단)	종이 계산으로 실험 대체
9	캘리브레이션 (LB 역산)	자유도 낮은 것부터
10	제출 (한 번에 하나만)	델타가 그 축의 순수 신호

11.2 상황별 처방

상황	먼저 볼 것
로컬은 오르는데 실전이 안 오름	7.7 전이비 → 7.1 폴드 설계 → 7.6 시드
새 모델을 만들지 말지 모르겠음	7.8 상한 스크린 (학습 전에)
블렌드가 안 오름	6.3 ρ . "다양성 키우기"는 레버가 아니다
후보가 너무 많음	7.5 무작위 대조군 + 닫힌형 판정
어떤 모델이 비정상적으로 좋음	8.3 특권 피쳐 확인
개선인지 우연인지 모르겠음	1.2 노이즈 바닥 + 7.6 5시드
제출했는데 0점	10.1 체크리스트 9항목
엔티티가 반복 등장함	4.2 이중 중심화 록업
누적·이동평균 컬럼이 있음	3.5 라벨 디코딩
시대·시즌이 나뉨	7.2 레짐 점검 → 분리 채점
캠페인을 접어야 하나	6.6 라이브러리 천장
GPU 를 쏘지 말지	10.2.1 — 상한 스크린 먼저. ρ 미달이면 켜지 마라
클라우드 GPU 가 불안정	10.2.3 — Secure 등급만, stop 대신 terminate

11.3 기법별 기대 이득 (이 대회 실측)

기법	이득	성격
as-of 분해 피쳐	+146.8	구조적
독립 파이프라인 결합	+38.5	구조적
신경망 1종 블렌딩	+36.2	구조적
엔티티 잔차 록업 (+대비 타겟)	+22.8	구조적
상태 복원	+17.1	구조적
제3 모델 추가	+7.6	구조적
아핀 캘리브레이션	+5.0	미세조정
블렌드 비율 조정	+1.7	미세조정
시드 수 증가	-12 ~ +2	미세조정

마지막 세 줄

1. 구조적 변경만 +15 이상을 낸다. 미세조정에 시간을 쓰는 것은 거의 항상 손해다.
2. 자유도가 기법의 운명을 결정한다. 정직한 검증을 통과해도 자유도가 크면 실전에서 죽는다.
3. 정직한 null 은 훌륭한 결과다. 이 대회에서 20+ 축을 빨리 닫은 것이 결과적으로 가장 생산적인 작업이었다. 억지로 성공을 만들면 슬롯과 시간을 잃는다.

실행 가능한 도구 — reusable_toolkit/ : 제출 0점 점검 · 행 독립성 감사 · 블렌드 계산기 (전부 실제 제출물로 동작 검증 완료)

기법 카탈로그 52종(채택 31 · 기각 17 · 보류 4) — playbook/ : python3 run.py list